Lecture 26: starting Deep Learning

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA7, WA7 out, due May 10
- Last class a week from today!
- Office hours continue (and end) next week!
- A note about the final
  - Questions similar in style to the quizzes: MCQ and fill in the blank
  - ~90 min, but we'll have 2 hours
  - You are allowed 1 side of a letter-sized piece of paper of handwritten notes

Deep Learning

Deep Learning roadmap

- Overview
- Neurons
- Perceptron
- Neural Networks
- Feed Forward
- Backpropagation

Overview of Deep Learning

Deep Learning vs Neural Networks

Deep Learning uses a particular kind of Neural Network as its **underlying structure**.

Deep Learning and Neural Networks are **not synonymous**.

What is Deep Learning?

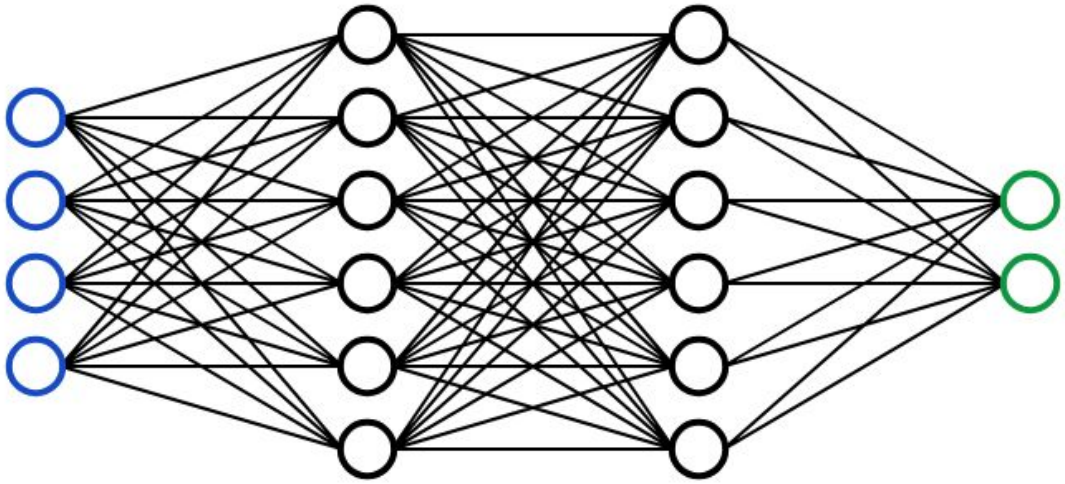
Deep Learning is a field of Machine Learning that teaches computers to process data in a way that is **inspired by the human brain**.

Deep Learning models recognize patterns in data, like complex pictures, text, and sounds, to produce accurate insights and predictions.

A **neural network** is the **underlying technology in Deep Learning.**

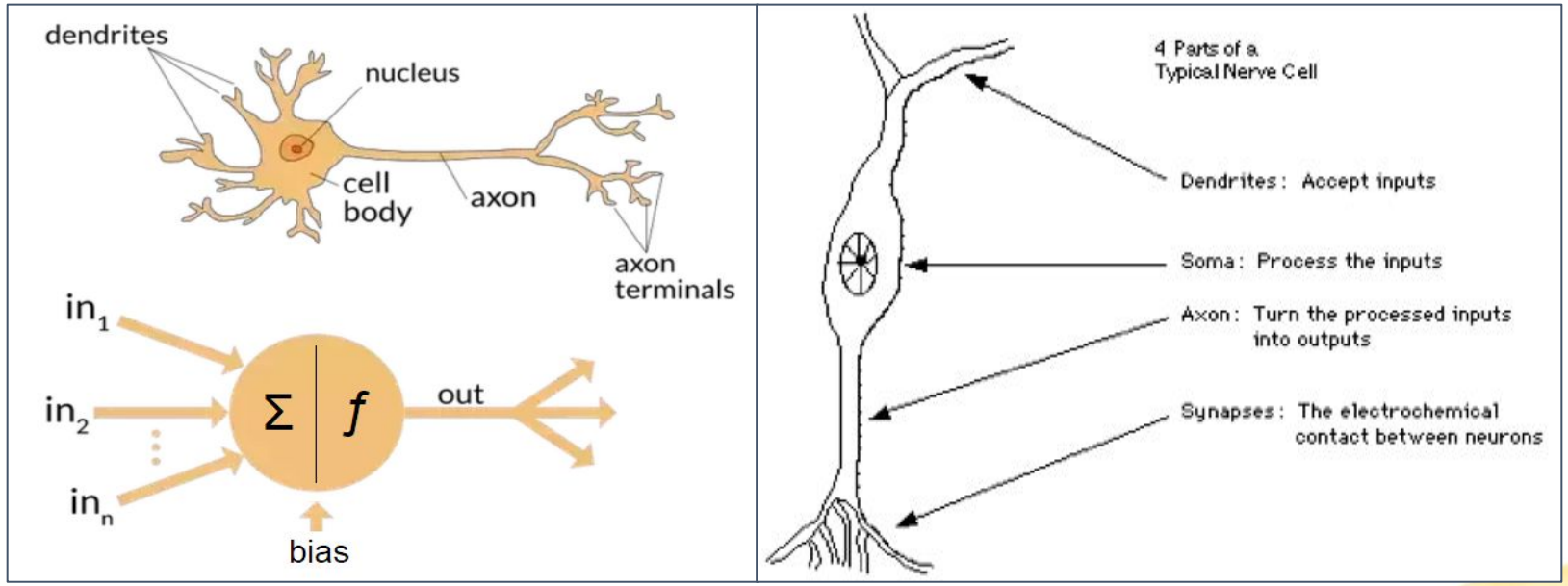
What is a Neural Network (NN)?

A neural network is a computational model in artificial intelligence inspired by the structure and functioning of the **human brain**.



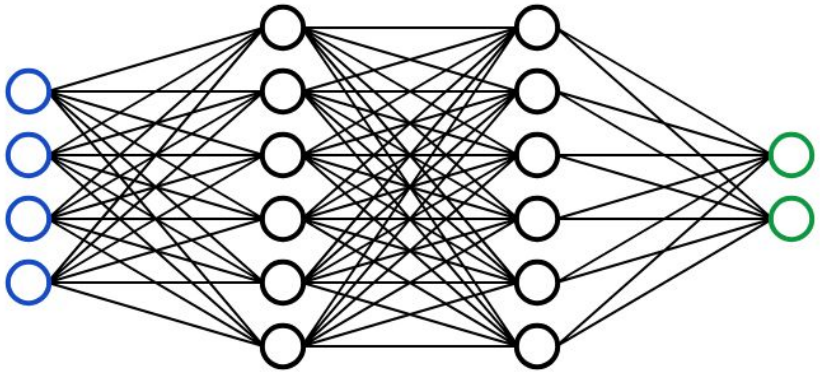
What is a Neural Network (NN)

A neural network is a learning model that uses **layer(s)** of **interconnected nodes** or neurons that resemble the human brain.



What is a Neural Network (NN)

It processes input data to make predictions/decisions by learning from the data, solving complex problems like document summarization, pattern recognition, and facial recognition with greater accuracy.



Neural networks consist of interconnected nodes, known as neurons, arranged in layers.

What makes Deep Learning “deep”?

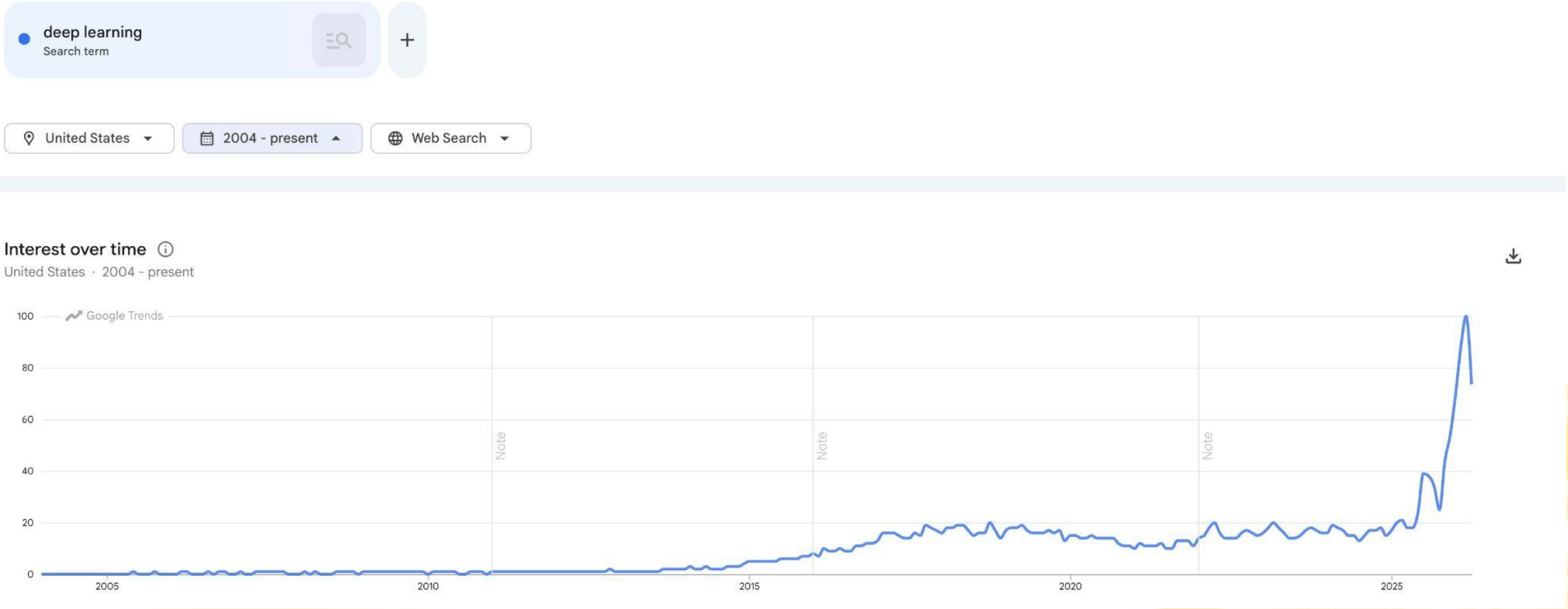
“Deep” refers to the **number of layers** in the network (*specifically* ***hidden layers***).

In a sense you’re going deeper by going through more network layers.

Applications of Deep Learning

- Face recognition
- Voice recognition
- Virtual assistants
- Medical surgery
- Robotics
- Self-driving cars
- …many more

Google Trends



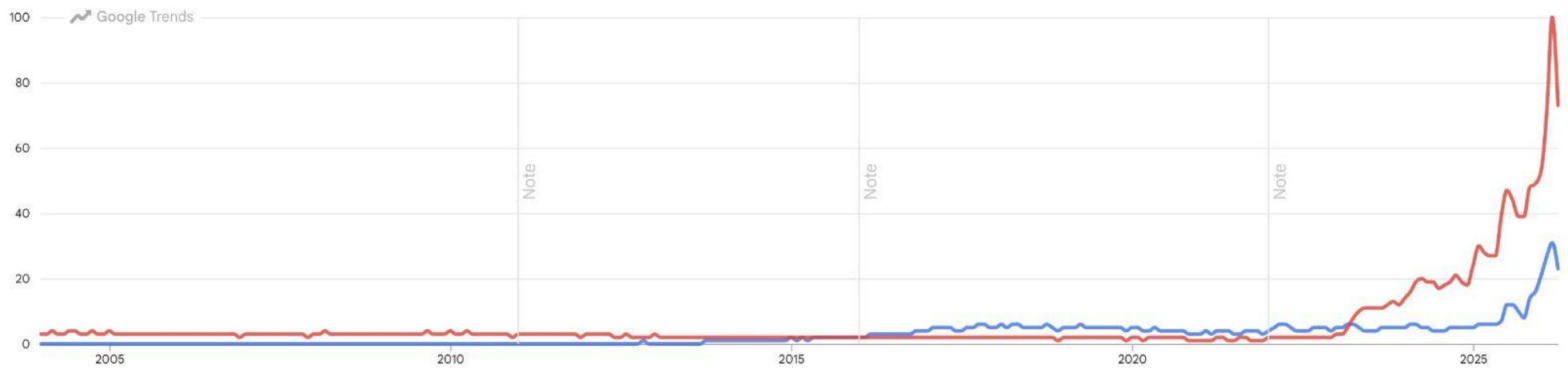
Google Trends

deep learning
Search term

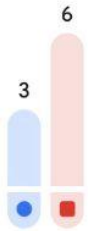
llm
Search term

United States | 2004 - present | Web Search

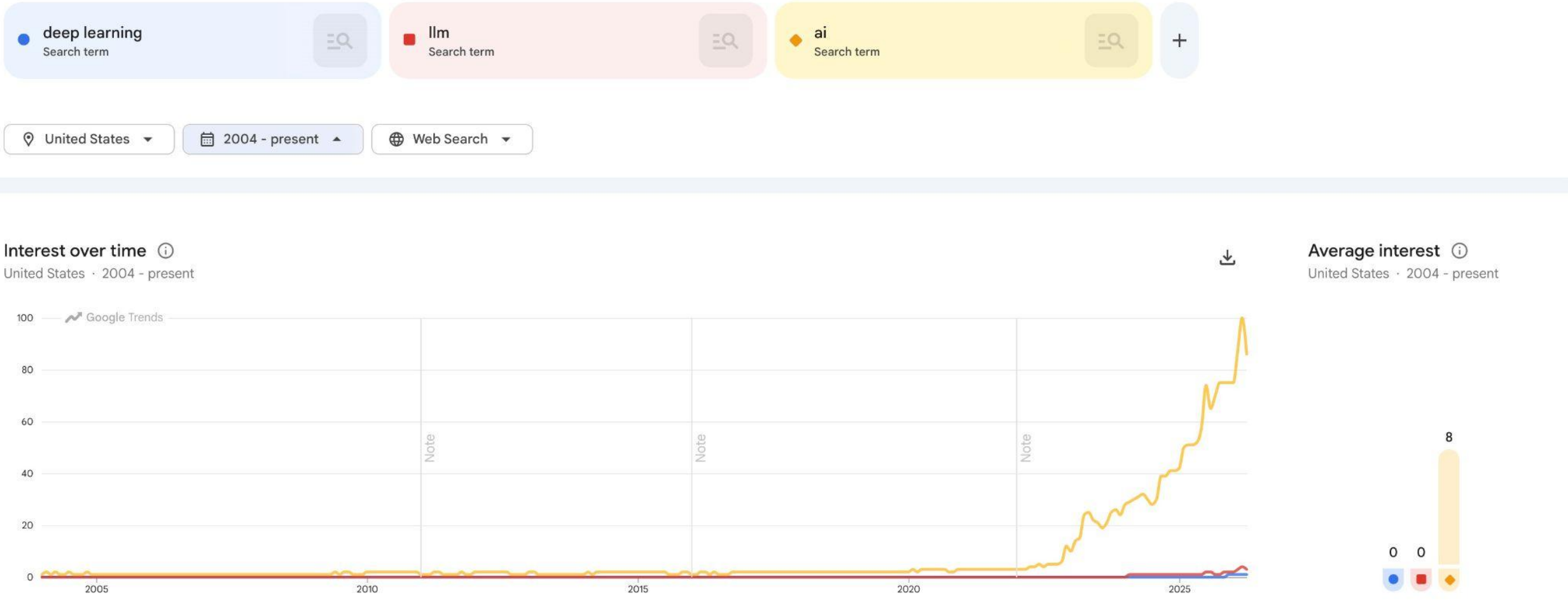
Interest over time
United States · 2004 - present



Average interest
United States · 2004 - present



Google Trends



Why Now?

The reasons for the explosion of Deep Learning are similar to the reasons for the explosion of Data Science.

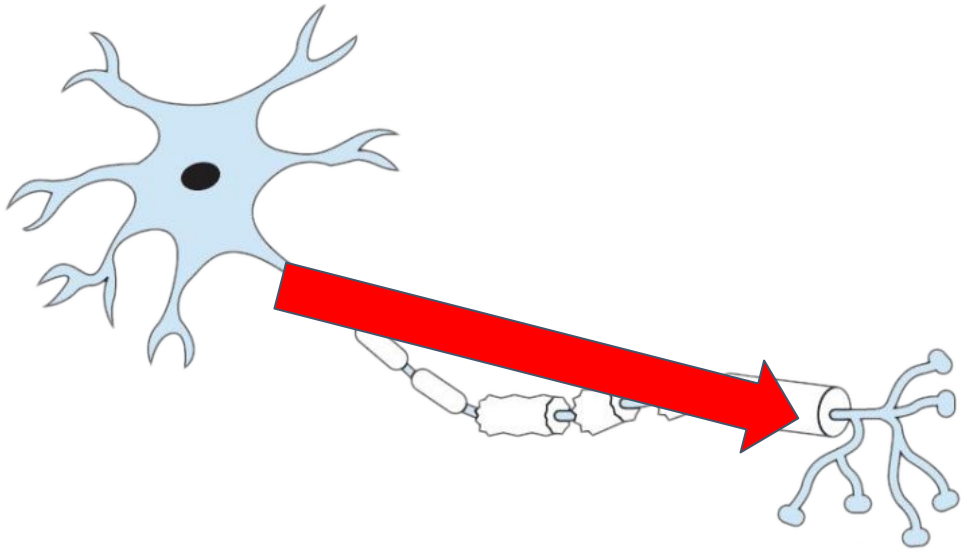
For example:

- Massive dataset abundance
- Better algorithms (ex. improved accuracy)
- More and cheaper compute power (ex. availability of cheap GPUs, cloud computing)
- Open Source tools and models

Building Blocks

Inspiration: Biological Neurons

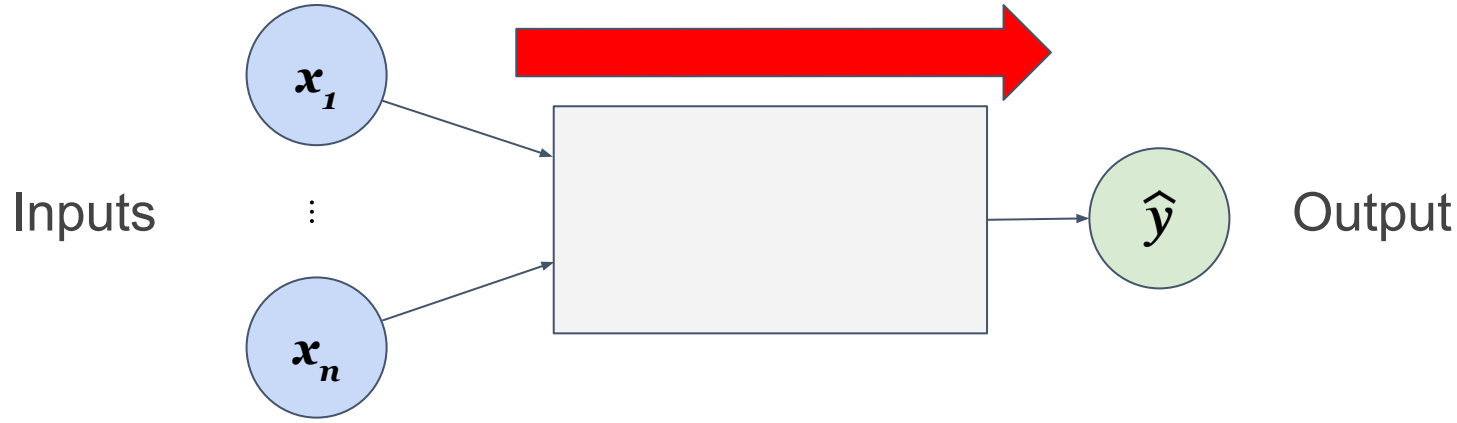
Neurons in animals/humans are the building blocks of how we compute.



Neurons take in input signals, and when those signals exceed a threshold, the neuron “fires” (spikes) and creates an output signal.

Building Blocks: Neurons

The basic unit of a neural network:



Neurons take **multiple input signals**, **process** them, and **produce an output**.

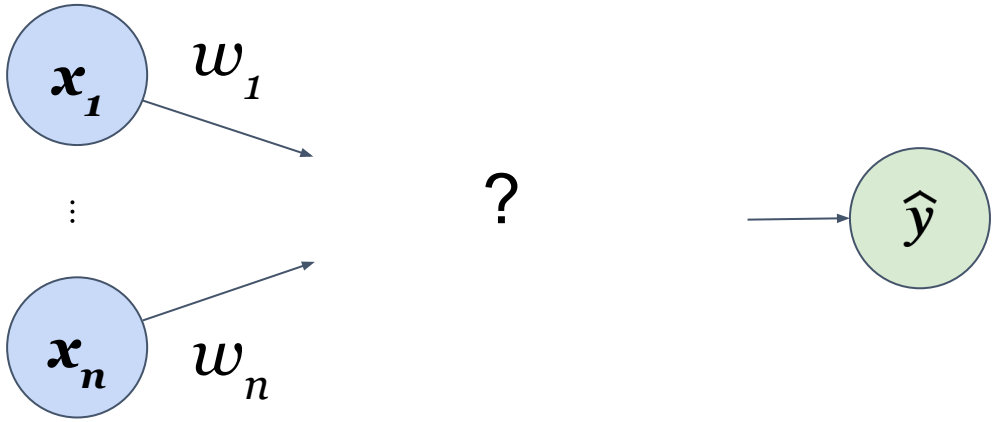
Building Blocks: Neurons

First, each input is multiplied by a weight, w

$$x_1 \rightarrow x_1 \cdot w_1$$

$$\vdots$$

$$x_n \rightarrow x_n \cdot w_n$$

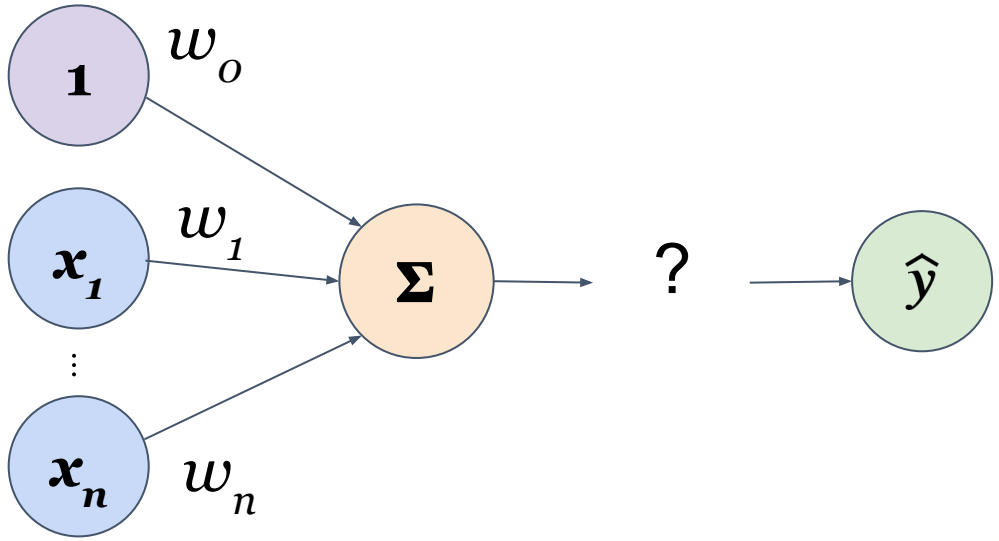


Building Blocks: Neurons

Next, an **input function** is applied.

This is usually adding all of the weighted inputs together with a bias, w_o:

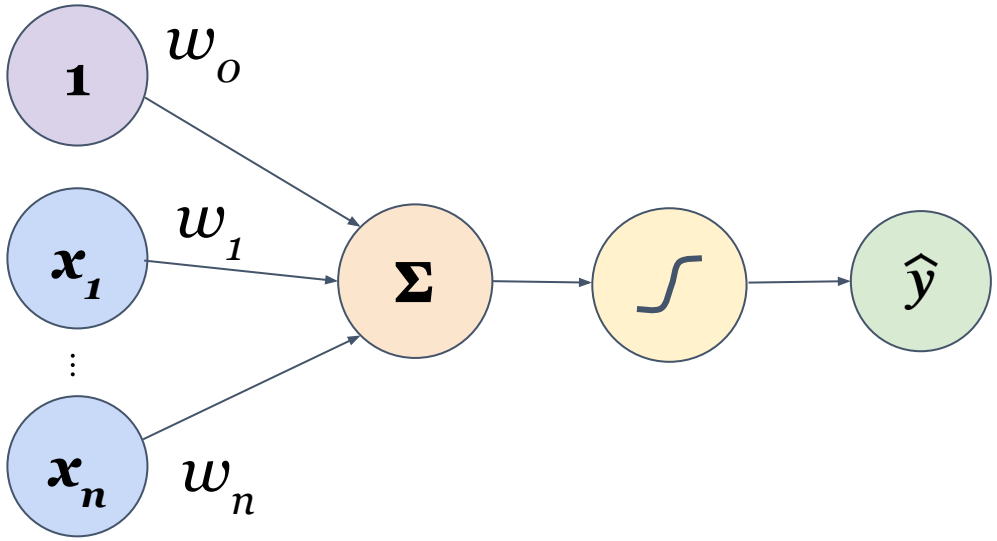
$$w_0 + \sum_{i=1}^{n} x_i w_i$$



Building Blocks: Neurons

Finally, the sum is passed through an **activation function**, f:

$$\hat{y} = f\left(w_0 + \sum_{i=1}^{n} x_i w_i\right)$$



Building Blocks: Neurons

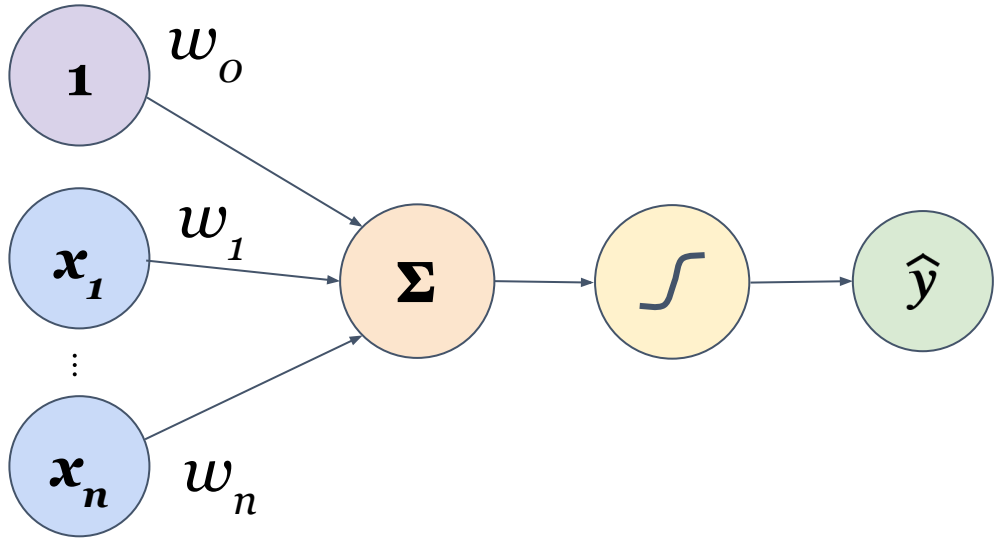
$$\hat{y} = f\left(w_0 + \sum_{i=1}^{n} w_i\, x_i\right) = f\left(w_0 + W^T X\right)$$

Output

Activation Function

Bias

Linear Combination of Inputs



Example

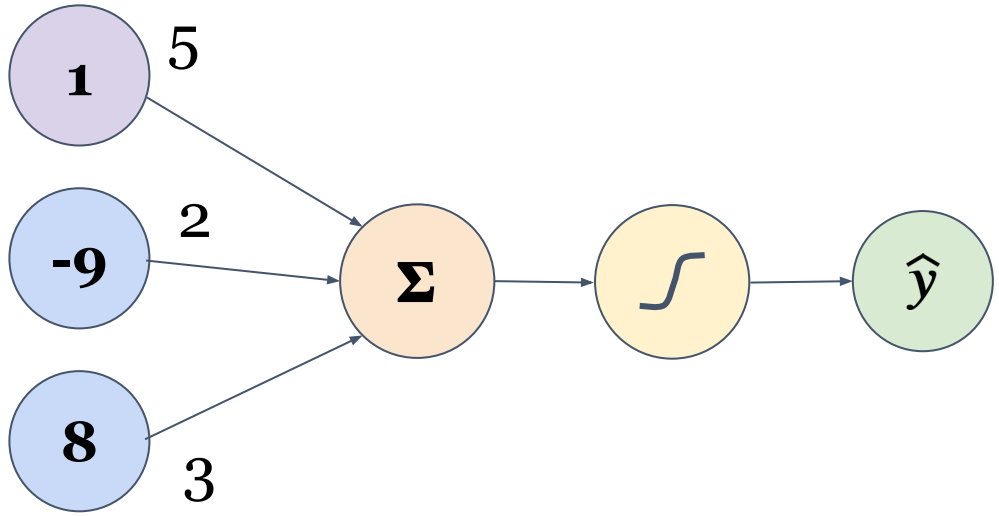
$$\hat{y} = f\left(w_0 + \sum_{i=1}^{n} w_i\, x_i\right) = f\left(w_0 + W^T X\right)$$

Output

Activation Function

Bias

Linear Combination of Inputs



Inputs Weights Sum Activation: Output

1 if x>0;
-1 if x<=0

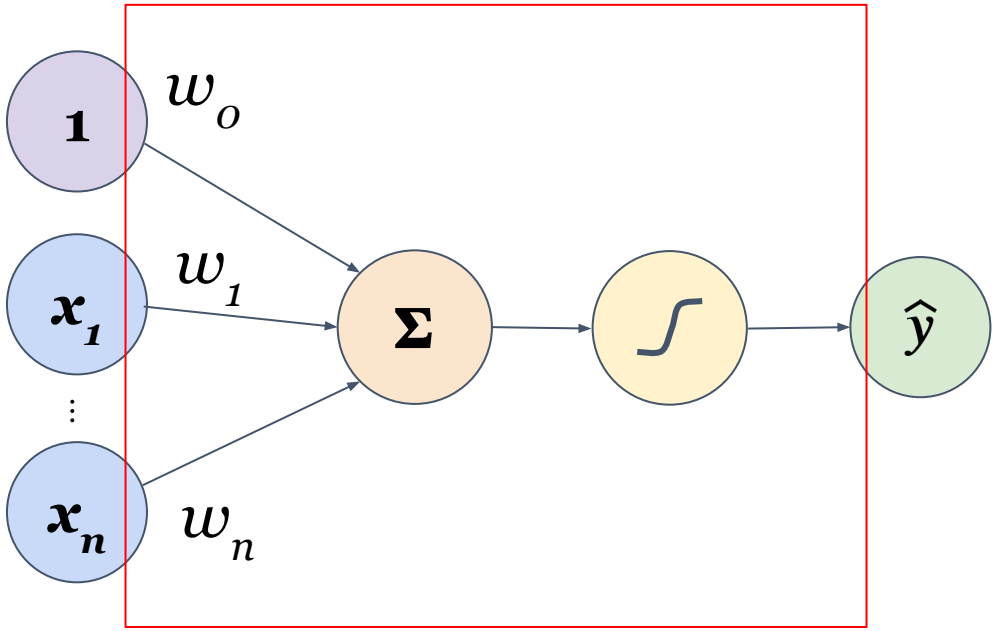
Building Blocks: Neurons

Let the linear combination of inputs plus the bias be denoted z.

$$z = w_0 + \sum_{i=1}^{n} x_i w_i$$

$$\hat{y} = f(z) = f\left(w_0 + \sum_{i=1}^{n} w_i x_i\right) = f\left(w_0 + W^T X\right)$$

A neuron contains

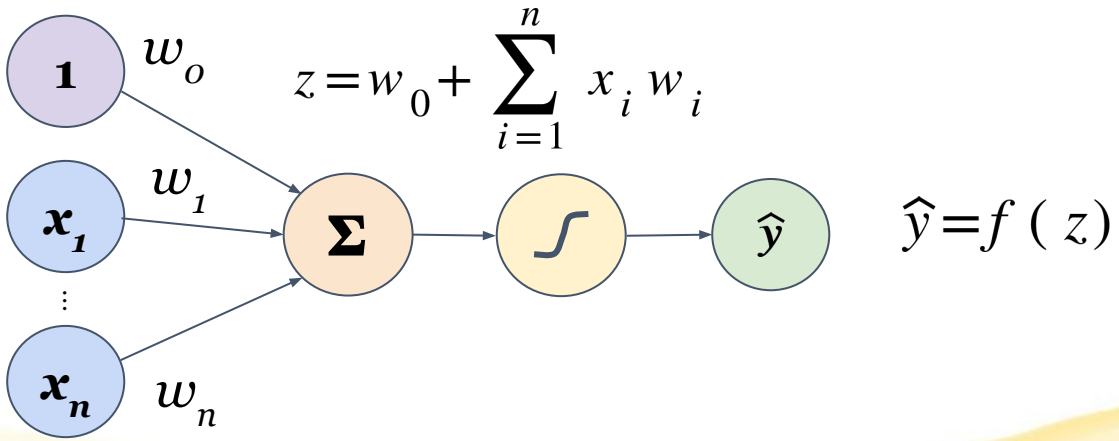


Perceptron

Early Neural Networks: The Perceptron

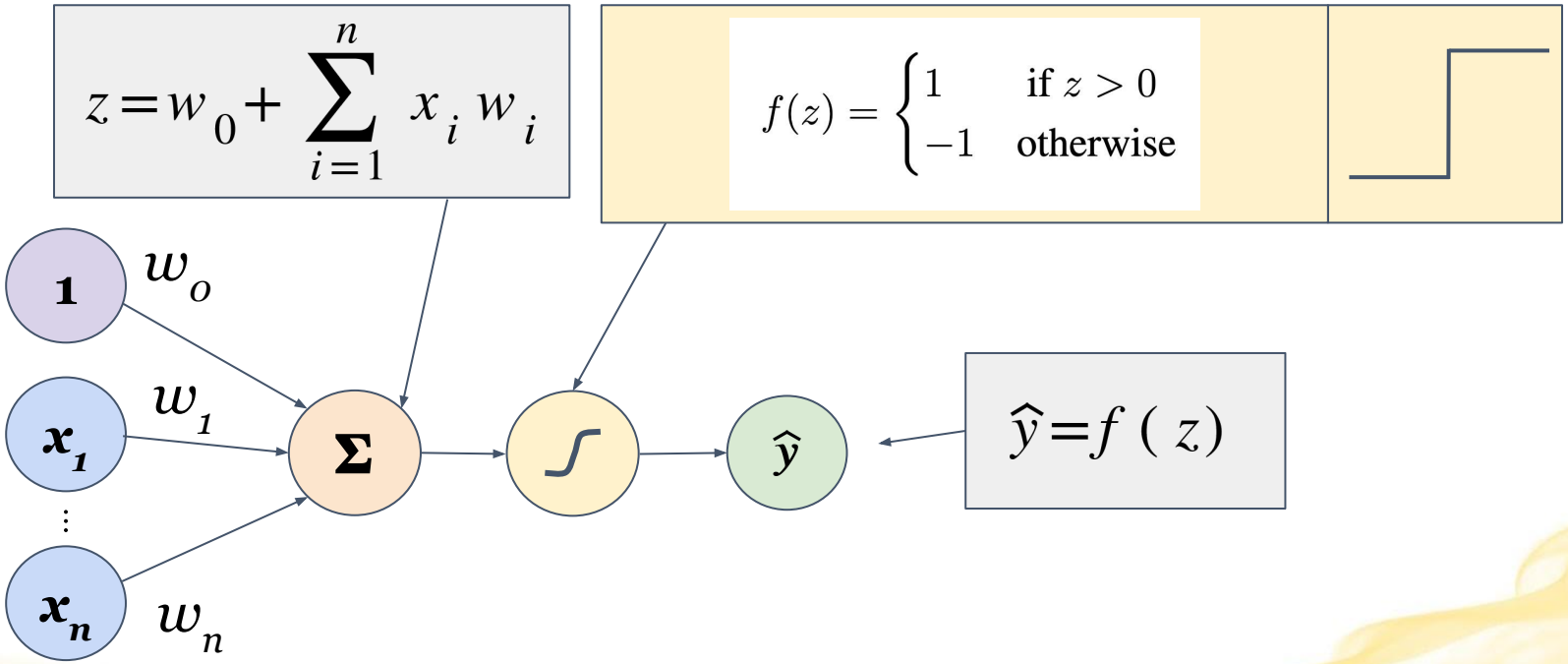
A perceptron is a type of Artificial Neural Network (ANN) and the simplest form of a neural network used for binary classification tasks.

It was introduced by Frank Rosenblatt in 1957 and consists of one neuron with some specificity and a learning mechanism.



Early Neural Networks: The Perceptron

The early perceptron used a simple linear threshold model (step rule) as its activation function, f.



Early Neural Networks: The Perceptron

The perceptron algorithm is a linear classifier that takes a multi-dimensional, real-valued input X, and learns a series of weights, W.

After learning, an input is classified as **positive** if z is positive, and is otherwise classified as **negative**:

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ -1 & \text{otherwise} \end{cases}$$

$$z = W^T \cdot X = w_o + w_1 x_1 + \ldots w_n x_n > o$$

Motivation for Bias

Notice that z represents an equation for a line (or hyperplane in n-dimensional space).

The motivation for needing a bias is that without it, this line/hyperplane must always pass through the origin.

$$z = W^T \cdot X = w_o + w_1 x_1 + \ldots w_n x_n > o$$

Conceptual Perceptron Algorithm

1. **Initialization:** Initialize the weights (including bias) to small, random values.
2. **Training**: For each training example (X, y), compute a predicted output $f(W^T \cdot X)$ using the current weights (W).
   - Update the weights (linear boundary) with each new training example:

$$w_j = w_j + \Delta w_j$$

3. **Repeat** Step 2 until the boundary isn't updated for any points

Training: Perceptron Learning Process

The algorithm iterates through all training examples, adjusting the weight vector with each iteration.

- Adjust the weights based on the **error between the predicted output and the actual target**.
  - This is usually done using the **Perceptron Learning Rule** (PLR), a simple form of gradient descent.
- Whenever a mistake in classification occurs, update the weights to attempt to correctly classify examples and converge towards the correct decision boundary.

The Algorithm: Perceptron Learning Rule

If the **prediction is wrong** (the model has misclassified that example), **update the weights**.

If the **prediction is correct** (same as the true/target value y), **don't update the weights**.

A prediction can be wrong in 2 ways:

1. False Positive
2. False Negative

The Algorithm: Perceptron Learning Rule

If $y \neq \hat{y}$ (mistake):

- $\Delta w_j = \alpha\, y\, x_j$; $\Delta w_o = \alpha y$ since $x_o = 1$
  - α = the **Learning Rate** (hyperparameter between 0 and 1)
  - If $\hat{y} = -1$ (predicting negative), but $y = +1$ (true label is positive)
    - **False Negative**
    - $y = +1$
  - If $\hat{y} = +1$ (predicting positive), but $y = -1$ (true label is negative)
    - **False Positive**
    - $y = -1$

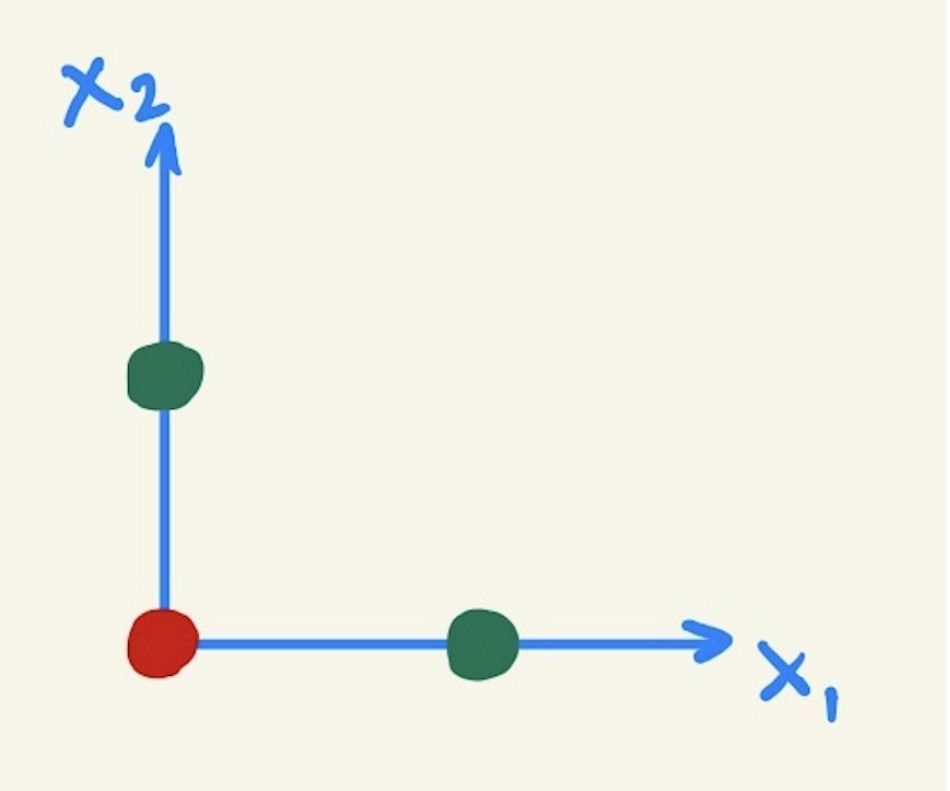
Algorithm example

(0,0,-1)

(0,1,+1)

(1,0,+1)

initial weights (0,0,0), learning rate 0.5



(see Perceptron Example file in Modules)

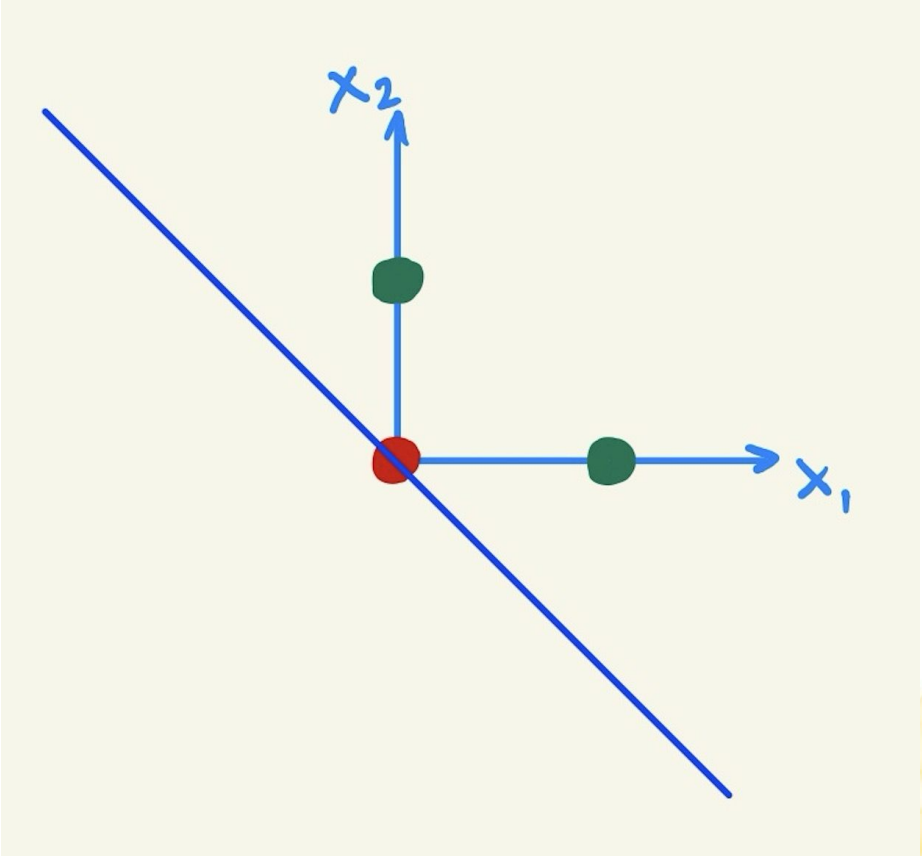
Algorithm example end

(0,0,-1)

(0,1,+1)

(1,0,+1)

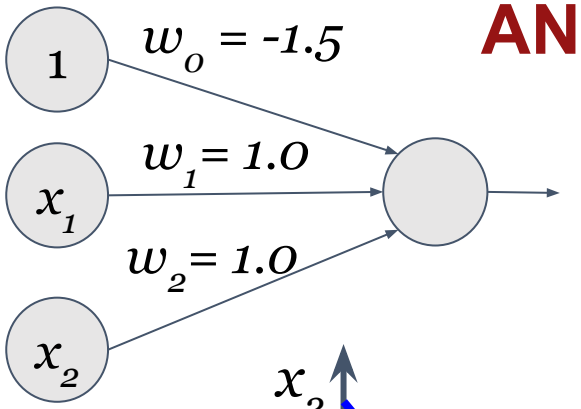
final weights (0, 0.5, 0.5) which corresponds to $0.5x_1+0.5x_2=0$, or $x_2=-x_1$



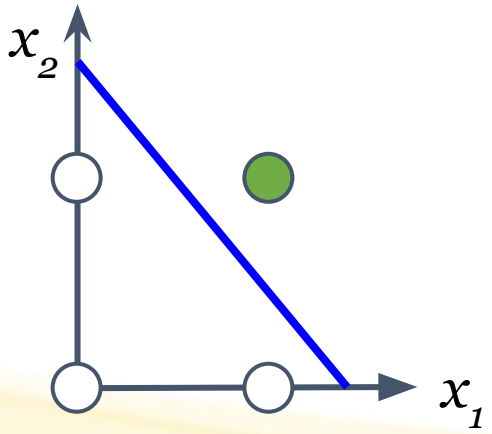
Expressiveness of Perceptrons

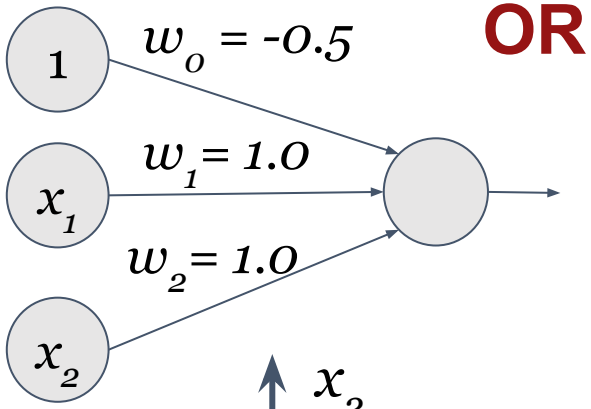
Using the step function for our activation function as before, we can represent a number of boolean expressions with Perceptrons.

Ex.

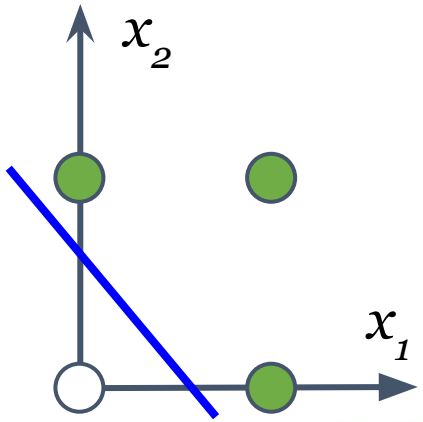


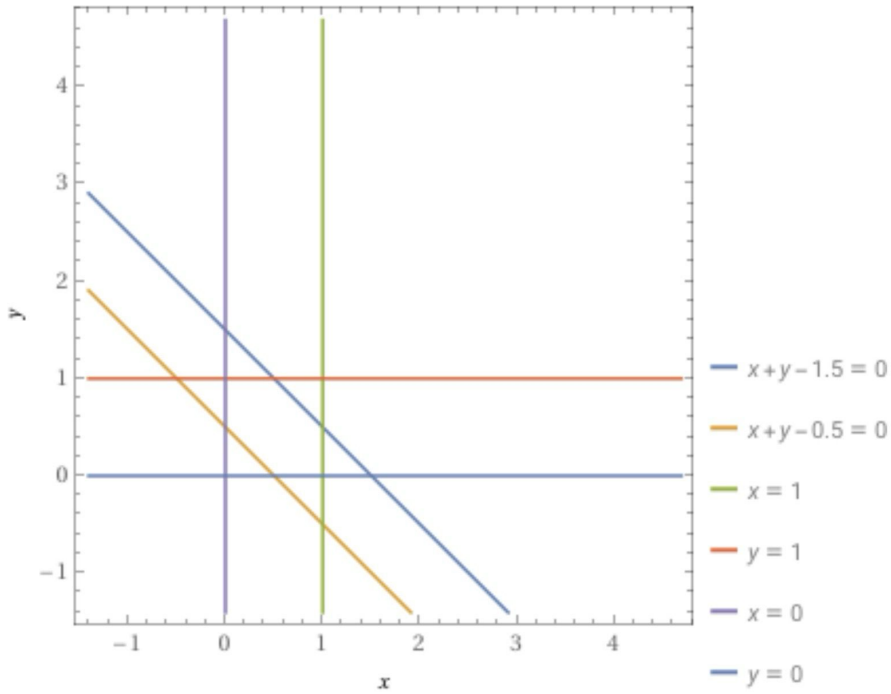
AND





OR



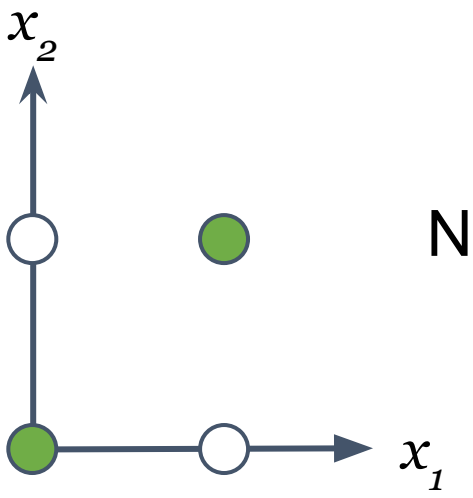


Expressiveness of Perceptrons

However, we are limited to those boolean expressions which can be separated linearly.

Consider:

XOR



Not linearly separable.

Limitation of Perceptrons

The perceptron algorithm had a number of weaknesses. Primarily, it could only classify **linearly separable data**.

- **Lacked hidden layers**, limiting its ability to represent complex relationships in the data.
  - It could not learn more intricate patterns or hierarchical features.
- It could only learn and model **linearly separable patterns**.
  - For non-linear data, it fails to converge to a solution (it can never separate data that are not linearly separable).
- Used only for **Binary Classification** problems.

Building on this

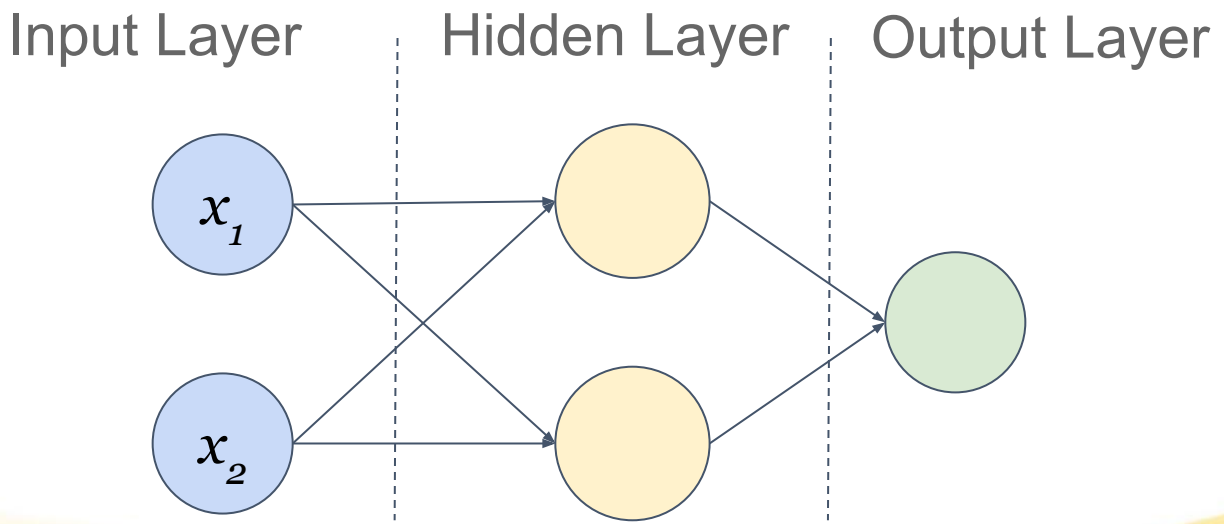
- Combine neurons in some way
- Use a more complex way to learn weights
- Use a more complex activation function

Neural Networks

Combining Neurons into a Neural Network (NN)

Neural Networks are networks of nodes connected by links.

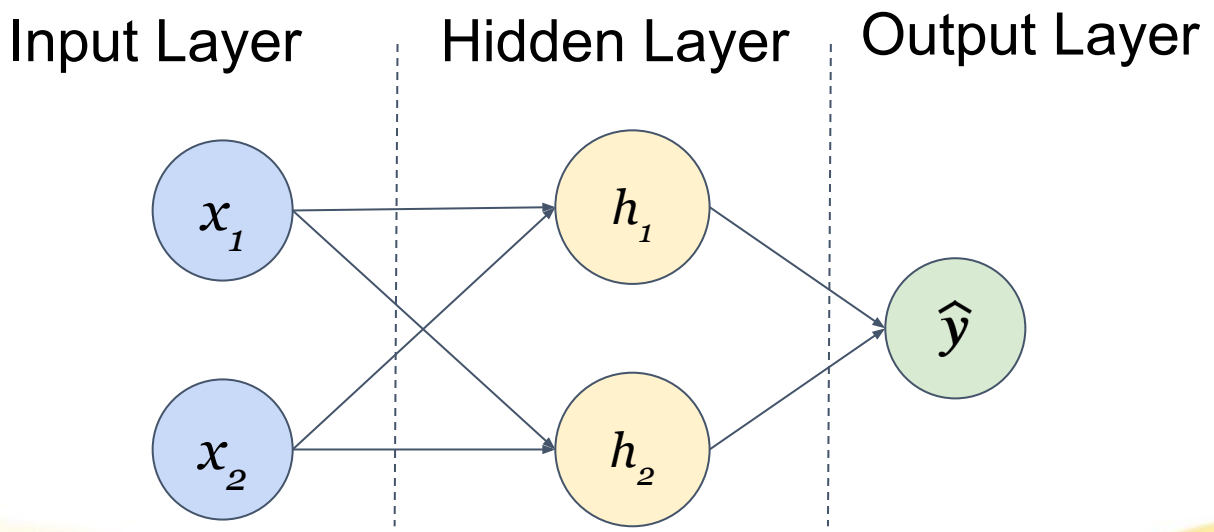
- A **hidden layer** is any layer between the input (first) layer and output (last) layer.
- There can be multiple hidden layers!



Combining Neurons into a Neural Network

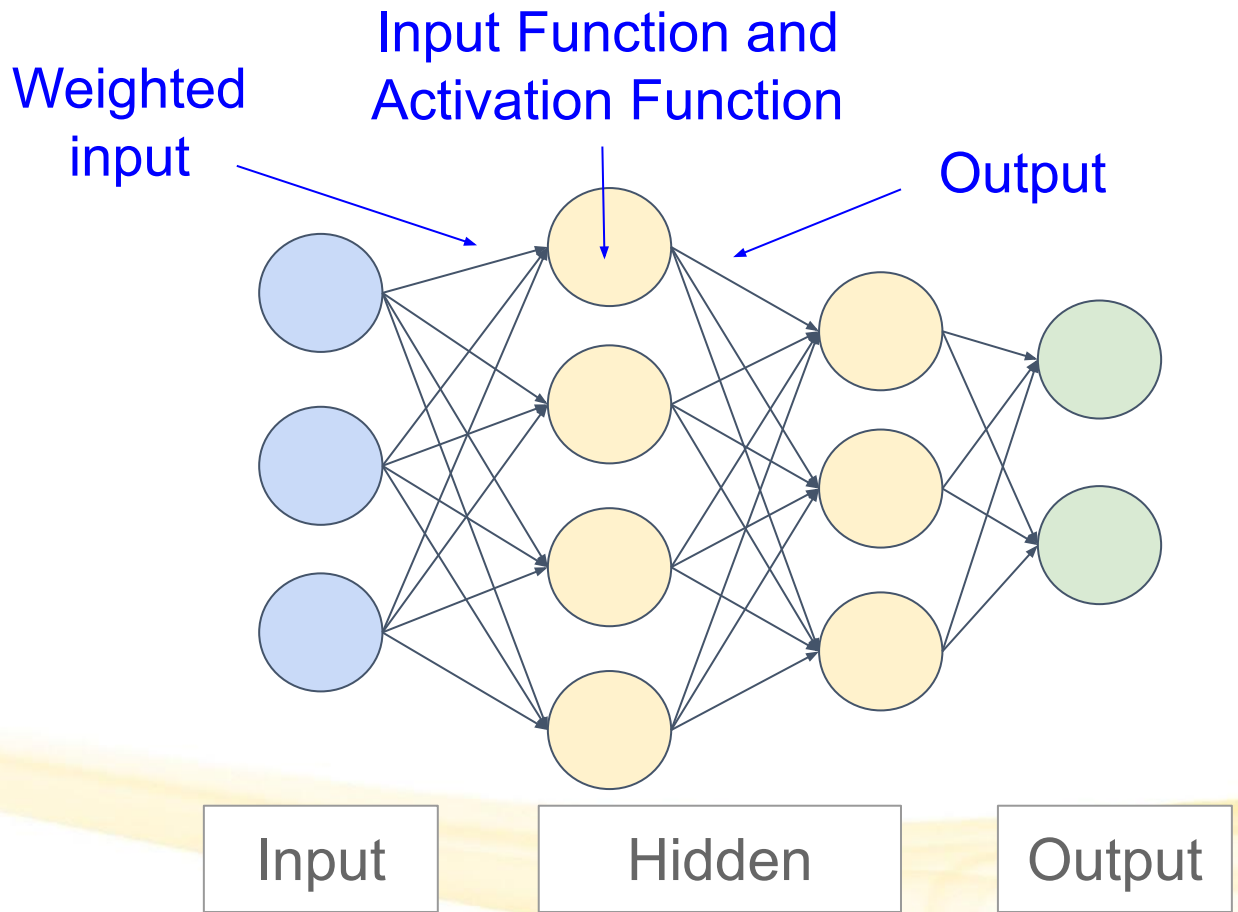
Example: This network has

- An input layer with 2 input nodes (x_1 and x_2)
- 1 hidden layer with 2 hidden nodes ($h_1 = f(z_1)$ and $h_2 = f(z_2)$)
- An output layer with 1 output node ($o_1 = \hat{y} = f(z_3)$).



Each Hidden Layer Node is a Neuron

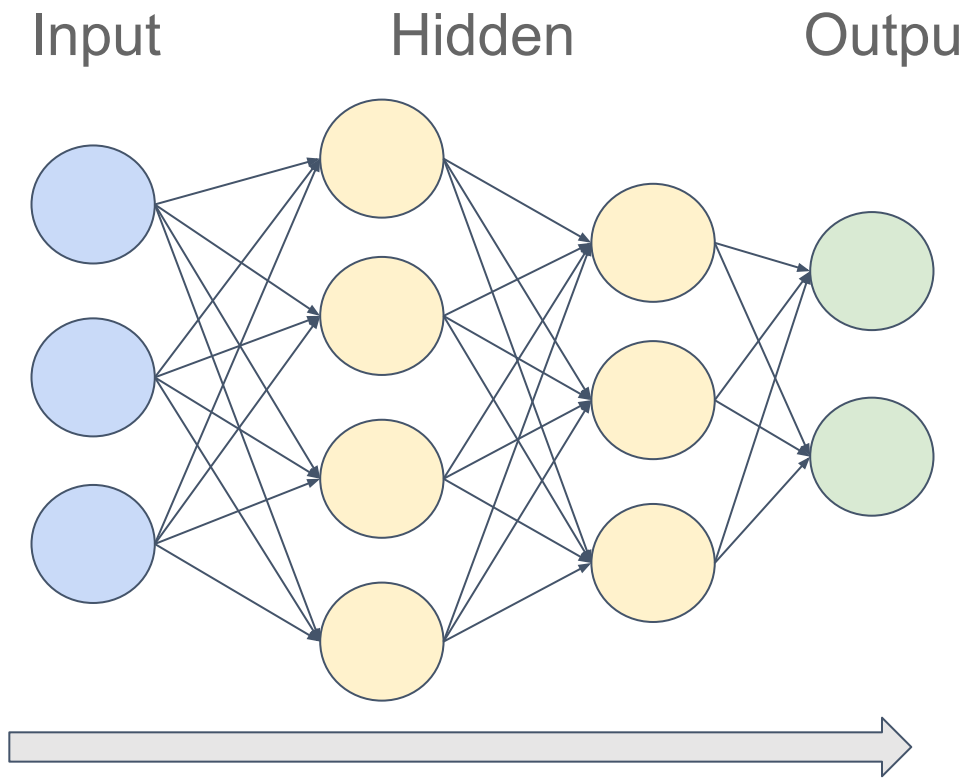
Each hidden node has an input with weights, an input function (ex. summing), an activation function, and an output.



Many deep neural networks are Feed-Forward

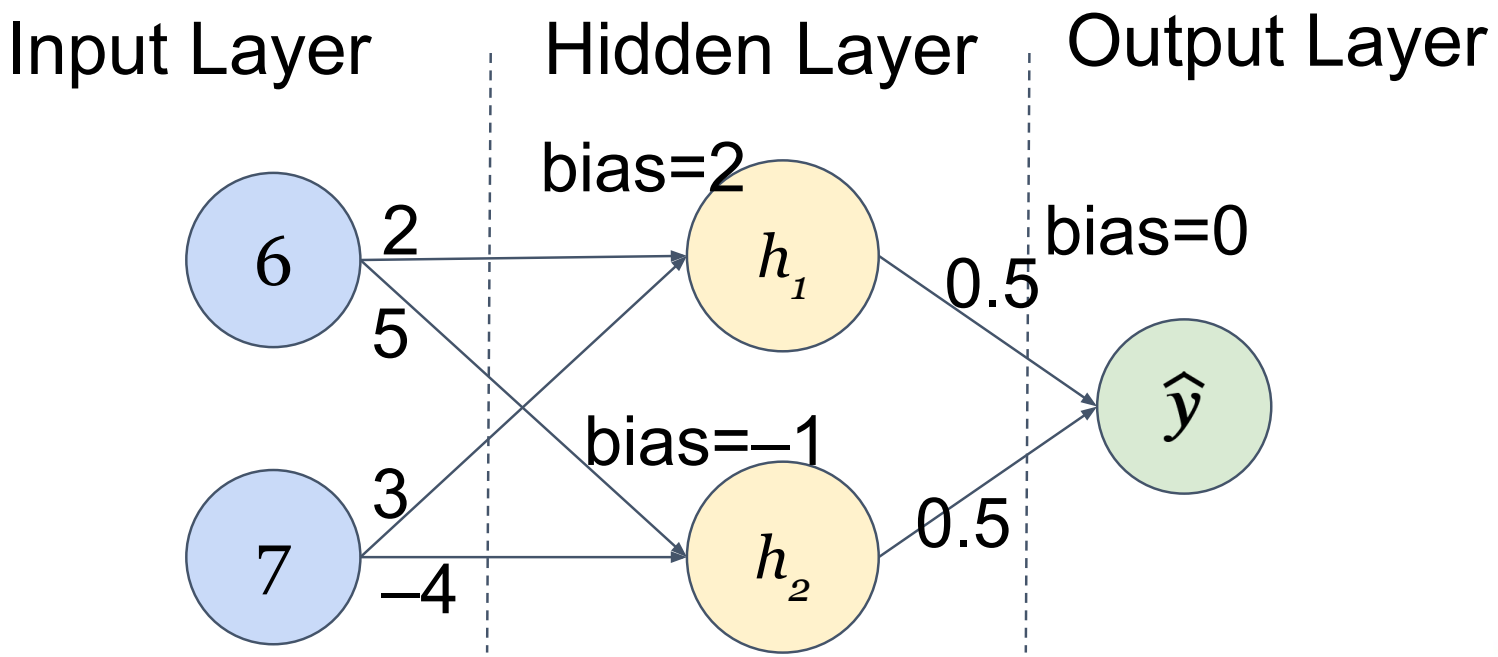
They only flow in **one direction**, from input to output.

The output of each layer is an input to the next layer.

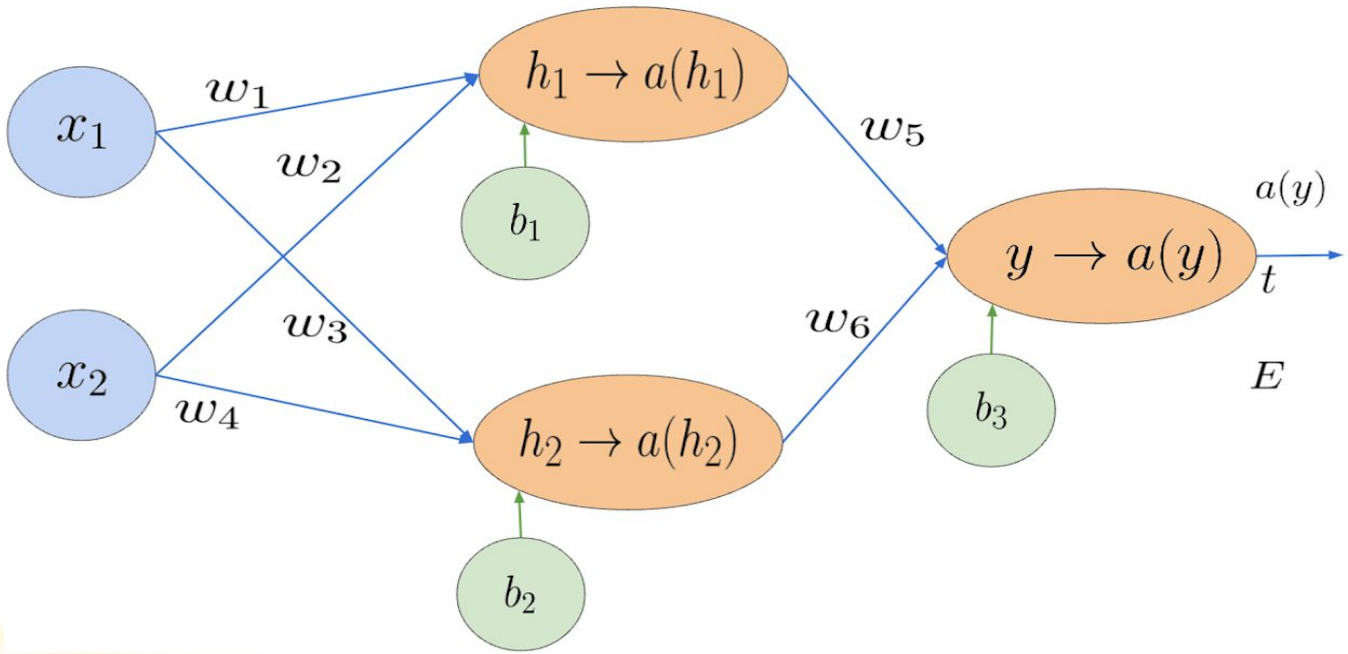


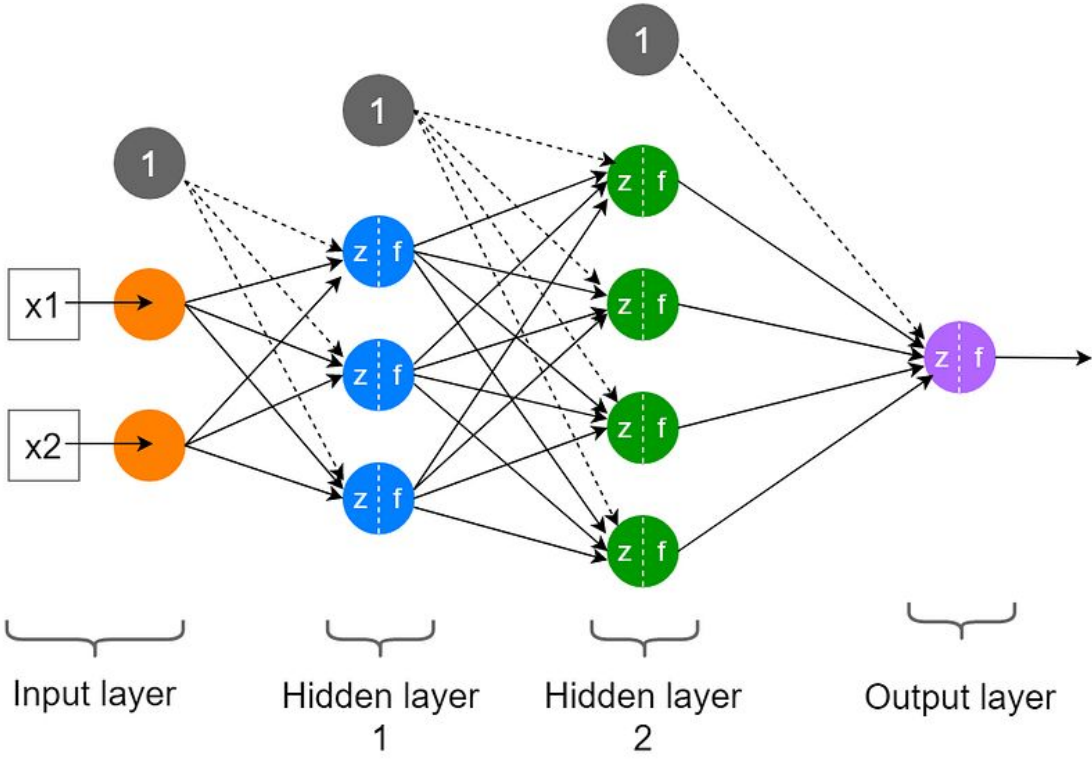
Feed Forward NNs are typically called **Artificial Neural Networks** (ANNs).

Example


Input Layer
Hidden Layer
Output Layer
bias=2
bias=0
bias=–1
6
7
2
5
3
–4
h_1
h_2
0.5
0.5
$\hat{y}$

Note on notation





Parameters in a neuron

Each node has an input function (typically summing over weighted inputs), an activation function, and an output.

- **Weights:** Learned during training; represent the **strength of connections** between nodes in different layers.
- **Bias:** Additional parameter in each neuron that is a constant added to the input of the activation function.
  - Helps adjust the output of each neuron alongside the weighted sum of inputs, even if all inputs are zero.
  - This term allows the activation function to **shift**, **affecting when it activates**.
  - Essentially, bias controls how easily a neuron activates based on inputs.

Both weights and biases are **learned during the training process**, adjusted iteratively to **minimize the difference** between predicted and actual outputs**.**

Multi-Layer Neural Networks

Multi-Layer Neural Networks

Multi-layer neural networks were invented after the Perceptron.

They had the following properties:

- Each computational unit, or neuron, is a node in a directed acyclic graph
- Neurons can have activation values, representing how activated one particular neuron is.
- Neurons can pass their activation values on to their neighbors, based on the weight of the connection between them.
- A multi-layer neural network usually has an input layer and an output layer.

Activation Functions

The activation function f acts as gate between the input to the neuron and its output. Akin to a biological neuron firing, it determines if the neuron will activate.

It operates on the inputs and weights from the previous layer (z).

$$f(z) = f\left(w_0 + \sum_{i=1}^{n} w_i x_i\right) = f\left(w_0 + W^T X\right)$$

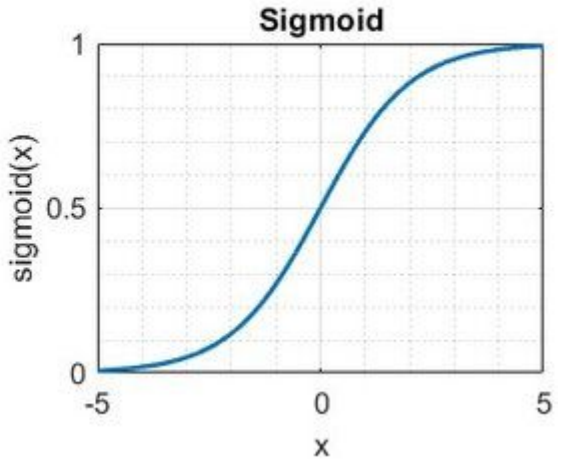
The simplest activation could be to simply multiply z by 0 or 1, acting as a switch that turns a node (neuron) in the network off or on.

.

Common Activation Functions

Sigmoid

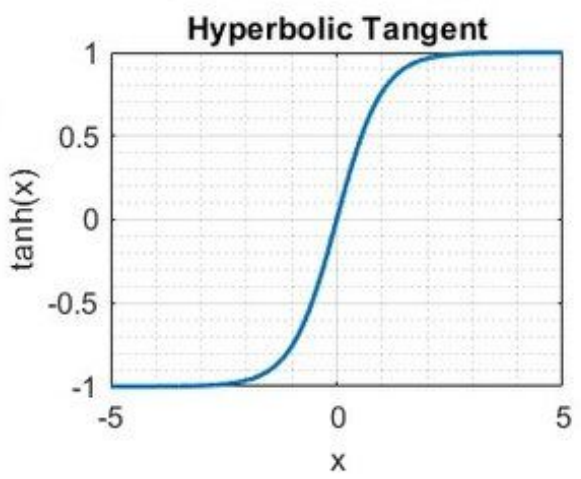
$$f(z) = \frac{1}{1 + e^{-z}}$$



Hyperbolic Tangent

(TanH)

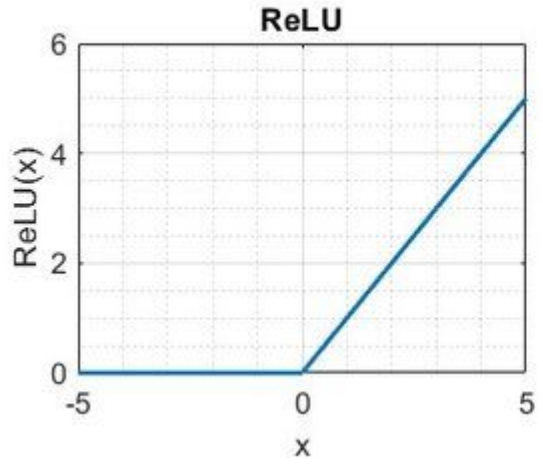
$$f(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



Rectified Linear Unit

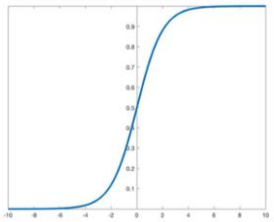
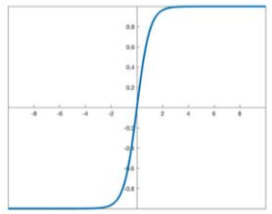
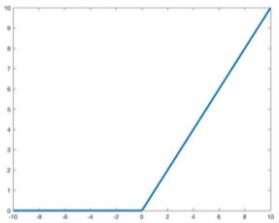
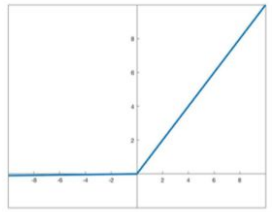
(ReLU)

$$f(z) = \max(0, z)$$



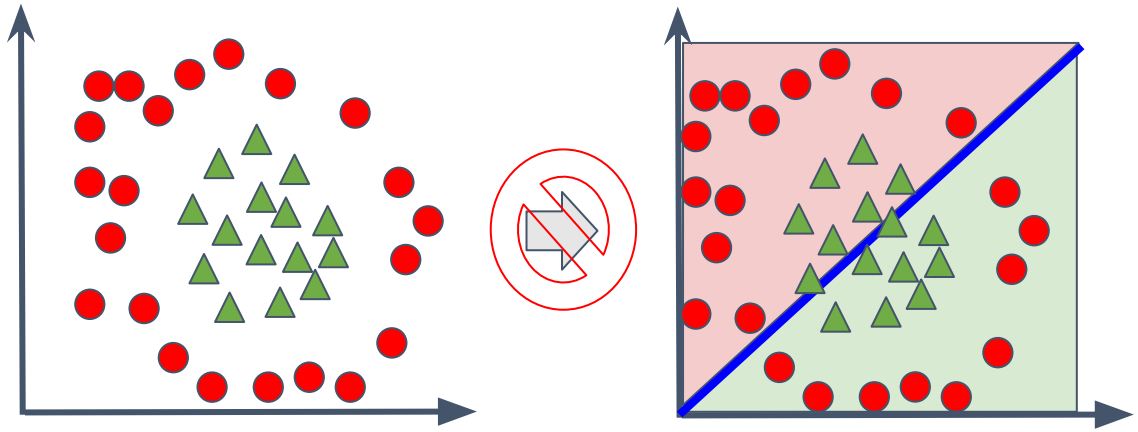
We don’t use a step function because it’s not suitable for Gradient Descent

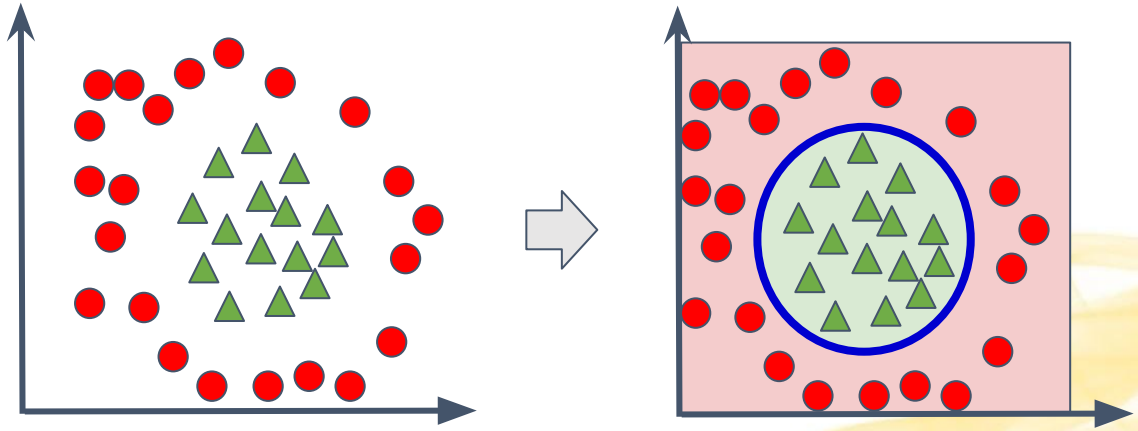
Comparing Activation Functions

Activation function	Equation	Graph	
Sigmoid	$S(x) = \frac{1}{1+e^{-x}}$		Outputs range from 0 to 1, suitable for binary classification tasks. Useful for probabilities.
Tanh	$\tanh x = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$		Outputs range from -1 to 1, providing stronger gradients for learning compared to sigmoid because the gradient is steeper.
ReLU	$RELU(x) = \begin{cases} 0 \text{ if } x < 0 \\ x \text{ if } x >= 0 \end{cases}$		Simple and computationally efficient. Sets negative values to zero (outputs x if x is positive, otherwise outputs 0.). Suffers from “Dying ReLU” problem.
Leaky ReLU	$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{otherwise} \end{cases}$		Addresses the Dying ReLU problem by allowing a small, non-zero gradient (ex. 0.01) for negative inputs.

Non-linearity

With multiple layers, we can model non-linearity





Output Activation

Depending on the type of problem, the output layer **may use a different activation function** than the hidden layers.

For example, when doing Regression with Neural Networks, it is common to apply a linear activation function at the very end (at the output).

- This is done to ensure the output is a continuous numerical value for our prediction without transforming it

Updating Neurons

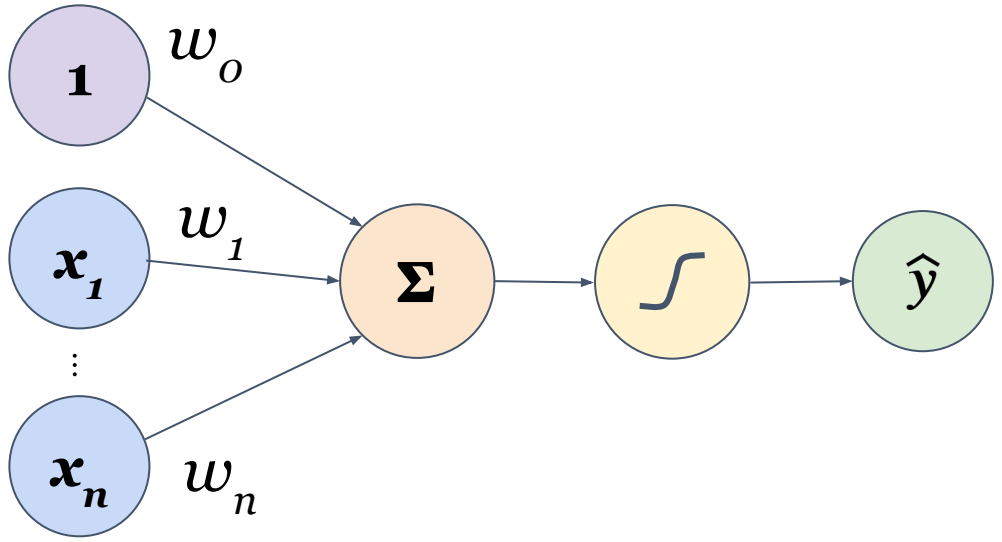
$$\hat{y} = f\left(w_0 + \sum_{i=1}^{n} w_i\, x_i\right) = f\left(w_0 + W^T X\right)$$

Output

Non-Linear Activation Function

Bias

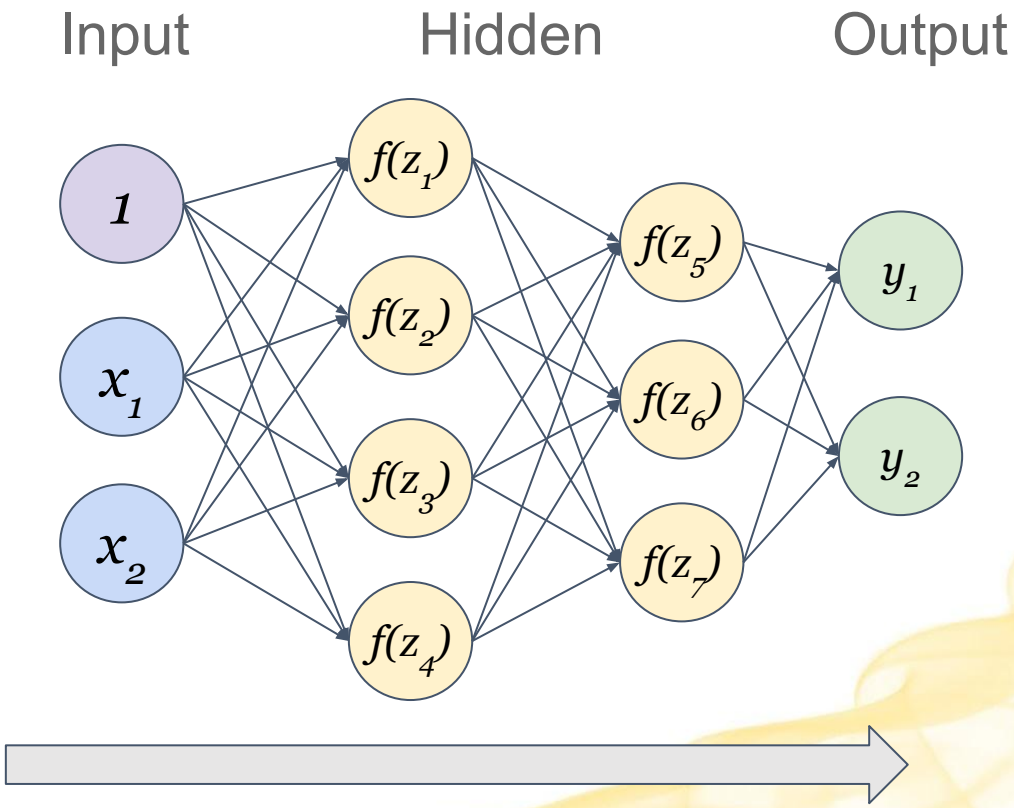
Linear Combination of Inputs



Inputs Weights Sum Non-Linear Activation Output

Multi-Layer Feed-Forward Neural Networks

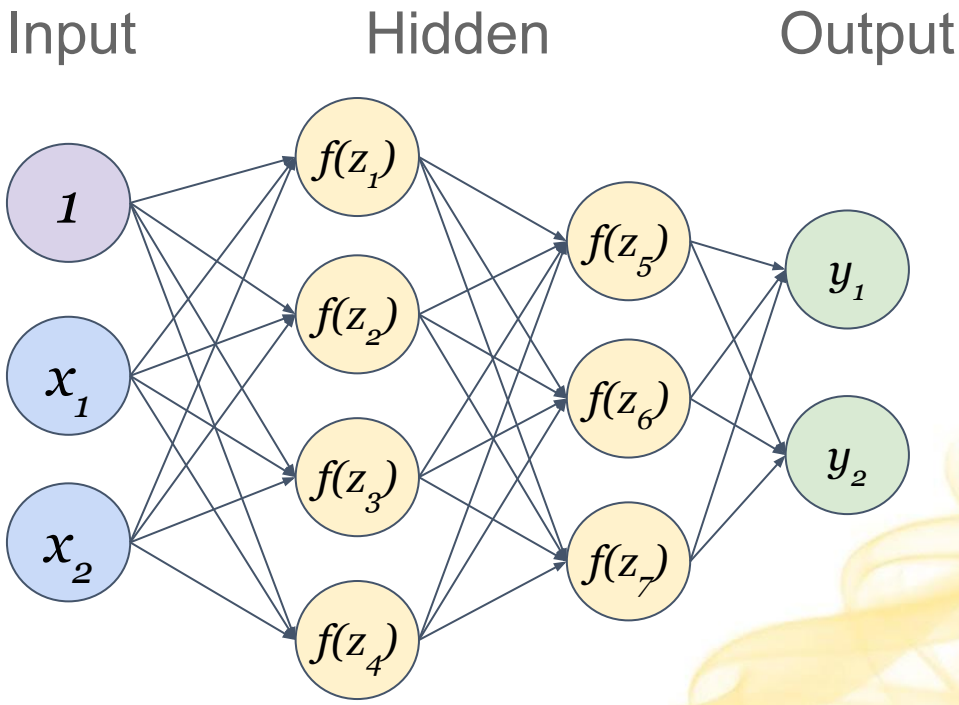
A type of ANN architecture that consists of multiple layers of interconnected nodes, with each layer passing information in a forward direction without any cycles (acyclic).



Multi-Layer Feed-Forward Neural Networks Mechanism

To classify something:

- Activate the input layer with the values of the input vector we want to classify.
- Propagate to each subsequent hidden layer
- Propagate to the output layer for a final prediction.

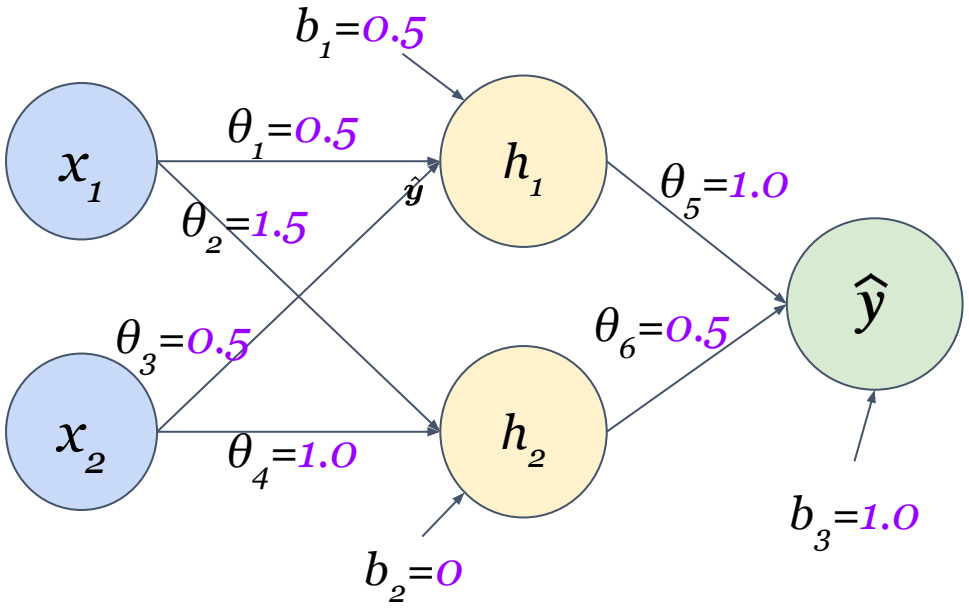


Another example

x_1=1, x_2=0

$$f(z) = \max(0,\ z) = \begin{cases} z & if\ z \geq 0 \\ 0 & otherwise \end{cases}$$

for all non-input nodes



confirm that you get 2.75